

Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Processamento de Linguagens

Ano Letivo de 2025/2026

Compilador Fortran 77

João Teixeira
A106836

Jorge Barbosa
A106799

Simão Mendes
A106928

May 09, 2026

PL

Índice

1. Introdução	1
2. Análise Léxica	1
2.1. O Formato Fixo (Fixed-Format)	1
2.2. Expressões Regulares em Detrimento de Literais Puros	1
2.3. Dissecção das Principais Expressões Regulares	2
3. Análise Sintática	4
3.1. Construção da Árvore de Sintaxe Abstrata	4
3.2. Resolução de Ambiguidade e Precedências	4
3.3. O Desafio dos Ciclos DO e Blocos Implícitos	5
4. Análise Semântica	6
4.1. Tabela de Símbolos e Gestão de Âmbitos	6
4.2. Sistema de Tipagem e Declaração Implícita	6
4.3. Validação de Controlo de Fluxo	6
4.4. Validação de Subprogramas	7
5. Otimizações	7
5.1. Simplificação de Expressões	7
5.2. Eliminação de Código Morto	7
5.3. Otimização de Saltos (<i>Jump Optimization</i>)	8
5.4. Dificuldades Encontradas	8
6. Geração de Código Máquina	9
6.1. A Lógica da Stack e Offsets de Memória (FP)	9
6.2. Tradução de Ciclos DO e Estruturas IF para Labels	9
6.3. Lógica de Pass-by-Reference	9
7. Conclusão	10
8. Referências Bibliográficas	10

Lista de Figuras

Figura 1 Exemplo de Estrutura de AST para Instrução Condicional Complexa	4
--	---

Lista de Tabelas

Tabela 1 Principais Expressões Regulares Estruturais do Analisador Léxico	2
Tabela 2 Hierarquia de Precedências na Avaliação de Expressões	5

1. Introdução

O presente documento relata o culminar do desenvolvimento de um compilador de Fortran 77, realizado no âmbito da unidade curricular de Processamento de Linguagens (2026). O “porquê” deste projeto não se resume apenas à avaliação académica, mas sim ao desafio intelectual de compreender as engrenagens internas de um tradutor de linguagens clássicas.

Fortran 77, sendo focado em computação numérica e contando com regras sintáticas rigorosas (como colunas fixas), obriga a uma abordagem léxica e sintática extremamente metódica. Em termos de ferramentas de suporte, a base de todo o trabalho assentou em **Python**, recorrendo ao pacote **PLY (Python Lex-Yacc)**. O `ply.lex` tratou da fase do *lexer*, enquanto o `ply.yacc` processou, regra a regra, a **gramática**.

Para executar o compilador e processar os ficheiros de teste, deve-se utilizar o módulo principal a partir da raiz do projeto. O comando base é o seguinte: **poetry run python src/main.py** Para executar o ficheiro de testes o comando é o seguinte: **poetry run python tests/test.py**

2. Análise Léxica

A primeira fase do compilador é responsável por ler o código fonte em Fortran 77 e transformá-lo numa sequência de símbolos lógicos (**tokens**). Esta etapa foi desenvolvida com recurso à biblioteca `ply.lex` da linguagem Python.

2.1. O Formato Fixo (Fixed-Format)

A linguagem Fortran 77, na sua norma original, impõe um formato posicional estrito. Para acomodar esta particularidade histórica, implementou-se uma função de pré-processamento (`tokenize`) que atua antes da análise léxica regular:

1. **Corte Posicional:** O código localizado para lá da coluna 72 é considerado comentário ou lixo segundo o padrão F77. O pré-processador corta as linhas exatamente nesse limite para garantir a conformidade.
2. **Continuação de Linhas:** Se a coluna 6 contiver um carácter diferente de espaço em branco ou do dígito zero, a linha atual é concatenada à linha de código anterior. Este mecanismo possibilita a quebra visual de instruções longas.
3. **Comentários:** Linhas cujo primeiro carácter seja `C`, `c`, `*` ou `!` são removidas na íntegra, e não chegam sequer a ser avaliadas pelo analisador léxico.

2.2. Expressões Regulares em Detrimento de Literais Puros

A biblioteca `ply.lex` permite capturar símbolos com recurso a duas estratégias distintas: a declaração de “literais puros” (através da lista `literals`) ou a definição de expressões regulares associadas a **tokens** nomeados.

No desenvolvimento deste compilador, a decisão arquitetural recaiu sobre a captura dos operadores (matemáticos, lógicos e relacionais) através de regras de expressões regulares. Esta abordagem resolve duas condicionantes essenciais:

1. **Limitações dos Literais Puros:** A variável `literals` do `PLY` apenas suporta a captura de caracteres isolados (por exemplo, `+`, `-`, `=`). A linguagem Fortran 77 possui múltiplos operadores compostos por vários caracteres, como é o caso da exponenciação (`**`), da concatenação de texto (`//`) ou dos operadores relacionais (como `.EQ.`).
2. **Insensibilidade a Maiúsculas e Minúsculas (Case-Insensitivity):** A norma do Fortran estipula que a linguagem é globalmente imune à capitalização (**case-insensitive**). Uma captura literal estrita impediria que o operador lógico `.AND.` aceitasse a variante `.and.`. Ao definir as expressões regulares com o modificador de agrupamento (`(?i:...)`), o motor léxico recebe a instrução para ignorar o tamanho das letras apenas naquela sequência específica.

Como consequência técnica desta escolha, os símbolos são empacotados em **tokens** providos de um nome interno abstrato (como `EQ` ou `AND`). Na fase posterior de análise sintática (no **parser**), esta abstração revela-se uma enorme vantagem ergonómica: permite referenciar estes terminais diretamente pelos seus nomes lógicos, sem necessidade de utilizar aspas (exemplo: referenciar `EQ` em vez de um literal confuso como `' .EQ. '`). Este requisito metodológico uniformiza e limpa a escrita das regras gramaticais em todo o compilador, independentemente de o programador ter escrito `.EQ.` ou `.Eq.`.

Um comportamento semelhante de normalização foi implementado para as palavras reservadas (`PROGRAM`, `INTEGER`, etc.). A expressão regular captura uma sequência alfanumérica genérica mas, antes da emissão do **token**, o seu valor é convertido para letras minúsculas e contrastado com um dicionário interno de palavras-chave. Desta forma, mantemos as expressões regulares lineares e erradicamos ambiguidades. Adicionalmente, atesta-se o limite de 6 caracteres imposto pelo Fortran 77 para os identificadores; se o tamanho exceder este teto, o compilador emite um aviso informativo no terminal.

É de notar que o repositório inclui o ficheiro `docs/lexer_full_keywords.py.bak`, que contém um analisador léxico preparado para suportar a especificação `PROLOG77` na íntegra. Contudo, para este projeto, optou-se por uma versão otimizada no ficheiro `src/lexer.py`, focada em garantir a compatibilidade com os exemplos de teste e em incluir as extensões consideradas fundamentais para a funcionalidade pretendida.

2.3. Dissecção das Principais Expressões Regulares

Para ilustrar de forma cabal o funcionamento do motor léxico, sumarizam-se na tabela seguinte as 6 expressões regulares estruturais formuladas no ficheiro `lexer.py`:

Categoria / Token	Expressão Regular (Regex)
Constantes Reais (<code>t_REAL_CONST</code>)	<code>(\d+\.\d* \.\d+)([EeDd][+-]?\d+)? \d+[EeDd][+-]?\d+</code>

Constantes de Texto (t_CHARACTER_CONST)	'(?:[^\'] '')*'
Operadores (ex: t_AND)	\.(?i:AND)\.
Identificadores (t_IDENTIFIER)	[A-Za-z][A-Za-z0-9]*
Comentários (t_COMMENT)	(?: [^] \n)[Cc][[^] \n]* (?: [^] \n)[*!][[^] \n]*
Constantes Lógicas (t_LOGICAL_CONST)	\.(?i:TRUE FALSE)\.

Tabela 1: Principais Expressões Regulares Estruturais do Analisador Léxico

Em seguida, detalha-se a mecânica de captura de cada um destes padrões:

1. **Constantes Reais e Dupla Precisão:** O padrão divide-se em duas alternativas principais separadas pelo traço vertical (|). A primeira metade aceita decimais tradicionais (\d+\.\d*|\.\d+), seguidos, de modo opcional, por um expoente científico ([EeDd], que acomoda o 'D' de dupla precisão), um sinal facultativo e os dígitos correspondentes. A segunda metade aceita inteiros puros que ostentem obrigatoriamente um expoente científico (\d+[EeDd]...). O símbolo 'D' é depois substituído por 'E' no código Python.
2. **Constantes de Texto:** Inicia e termina com plicas literais (''). O núcleo do padrão (?:[^\']|'')* estipula a leitura de qualquer sequência que não contenha plicas isoladas ([^']), mas aceita contudo plicas duplas ''. No Fortran, a plica dupla constitui o mecanismo natural de escape para introduzir uma plica no interior da própria cadeia de texto.
3. **Operadores Lógicos e Relacionais:** Os pontos de delimitação iniciais e finais são escapados através de barra oblíqua inversa (\.). O grupo interno (?i:AND) assinala com clareza que a validação das letras A, N e D deve ignorar a capitalização, o que valida com igual sucesso .AND., .and. ou .aNd..
4. **Identificadores (Variáveis):** Aplica a regra matricial de nomenclatura do Fortran, exigindo imperativamente que o primeiro carácter seja uma letra autêntica ([A-Za-z]). A partir dessa letra inaugural, permite uma sucessão indefinida (*) de caracteres alfanuméricos.
5. **Comentários (Regex de Limpeza):** Utilizada na primeira triagem para processar anotações. Encontra-se ancorada ao início do ficheiro ou a uma quebra de linha (?:[^]|\n). De seguida, exige os símbolos de demarcação ('C', 'c', '*', ou '!'). O último segmento [[^]\n]* consome todos os caracteres à direita, até à interceção de uma nova linha.
6. **Constantes Lógicas (Booleanas):** Semelhante à abordagem metodológica dos operadores relacionais, exige os pontos de limitação obrigatórios e avalia, de forma alternada, as palavras reservadas 'TRUE' e 'FALSE'. A presença do modificador ?i: confere-lhe total imunidade a variações de letras maiúsculas ou minúsculas.

3. Análise Sintática

Após a conversão do texto numa sequência linear de **tokens**, a análise sintática é encarregue de validar a estrutura gramatical do programa e construir a Árvore de Sintaxe Abstrata (AST - **Abstract Syntax Tree**). Para tal, tirou-se partido da ferramenta `ply.yacc`, que concretiza um algoritmo de **parsing** ascendente (**Bottom-Up**) da família LR (especificamente da subfamília LALR(1)). A especificação completa da gramática, extraída diretamente das definições do código-fonte, pode ser consultada no ficheiro `docs/grammar.txt`.

3.1. Construção da Árvore de Sintaxe Abstrata

As regras da gramática do Fortran 77 foram transcritas para a linguagem Python na forma de funções de produção (denominadas `p_...`). Cada regra avaliada com sucesso instancia nós concretos da AST (como os nós `ProgramNode`, `IfNode`, ou `BinOp`), garantindo assim uma representação em memória estruturada, hierárquica e facilmente processável pelas etapas subsequentes do compilador.

A título de exemplo, considere-se a seguinte instrução complexa que envolve uma condição lógica, uma atribuição aritmética e uma chamada de função:

```
IF (VALOR .GT. 0) RESULT = X + FUNC(A, 1.0) * 2
```

Segues-se a sua representação conceptual em forma de árvore:

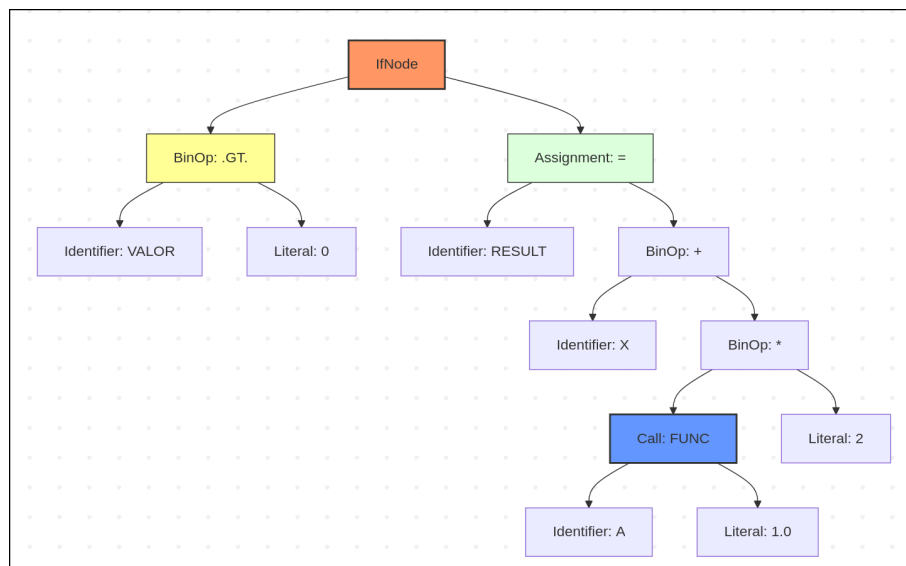


Figura 1: Exemplo de Estrutura de AST para Instrução Condicional Complexa

3.2. Resolução de Ambiguidade e Precedências

No intuito de lidar de forma robusta com as ambiguidades naturais associadas às expressões matemáticas, a biblioteca `yacc` permite a configuração explícita e declarativa

das precedências e associatividades. No contexto da linguagem Fortran, estipulou-se a seguinte hierarquia, ordenada da prioridade máxima para a mínima:

Prioridade	Operadores	Associatividade
1 (Mais Alta)	Exponenciação (**)	Direita
2	Multiplicação e Divisão (*, /)	Esquerda
3	Adição e Subtração (+, -)	Esquerda
4	Concatenação de Textos (//)	Esquerda
5	Relacionais (.EQ., .NE., .LT., .LE., .GT., .GE.)	Esquerda
6	Negação Lógica (.NOT.)	Direita
7	Conjunção Lógica (.AND.)	Esquerda
8	Disjunção Lógica (.OR.)	Esquerda
9 (Mais Baixa)	Equivalência Lógica (.EQV., .NEQV.)	Esquerda

Tabela 2: Hierarquia de Precedências na Avaliação de Expressões

3.3. O Desafio dos Ciclos DO e Blocos Implícitos

Contrariamente às linguagens de programação contemporâneas que recorrem a delimitadores explícitos para marcar o início e o fim de um bloco de código (tais como as chavetas {} em C ou Java), o Fortran 77 sinaliza o fim de um ciclo DO através da referência numérica a uma instrução rotulada (**label**), recorrendo na esmagadora maioria dos casos ao comando nulo CONTINUE.

Atendendo a que a ferramenta `ply.yacc` atua como um **parser** ascendente (LALR(1)), o mecanismo de resolução de blocos em tempo de execução gramatical revela-se complexo, pois os limites não estão sintaticamente confinados como aconteceriam num analisador descendente (**Top-Down LL**).

Para ultrapassar este obstáculo técnico, a solução concebida assenta num processo de duas fases:

1. Numa primeira etapa, aquando da passagem normal do **parser**, os nós correspondentes aos ciclos (DoNode) e as respetivas instruções de fecho (como o ContinueNode) são produzidos sequencialmente e arrumados numa lista plana de comandos.
2. Numa segunda etapa, após a conclusão total da análise gramatical de todo o subprograma, aciona-se um procedimento de pós-processamento (funções `_attach_do_blocks` e `_find_matching_do_terminator` em `parser.py`). Esta rotina inspeciona a lista plana da árvore, deteta o **label** numérico exigido pelo

DO inicial e procura a instrução com o rótulo correspondente (tipicamente o CONTINUE). Ao estabelecer essa ligação, todas as instruções compreendidas entre os dois pontos são encapsuladas e aninhadas no bloco de código interno pertinente do nó DoNode. Assim, assegura-se que a versão final da AST espelha corretamente a hierarquia lógica pretendida.

4. Análise Semântica

A fase de análise semântica atua sobre a **Árvore de Sintaxe Abstrata (AST)** gerada pelo analisador sintático, garantindo que o programa, embora gramaticalmente correto, obedece às regras de significado e tipagem da linguagem *Fortran 77*. Para este efeito, foi implementada a classe *SemanticAnalyzer*, que percorre a árvore recorrendo ao padrão *Visitor*.

4.1. Tabela de Símbolos e Gestão de Âmbitos

O núcleo da validação semântica é a *SymbolTable*. Esta estrutura foi implementada com suporte para múltiplos âmbitos (geridos através de uma pilha de dicionários, manipulada pelos métodos *enter_scope* e *exit_scope*), o que permite um isolamento adequado das variáveis locais pertencentes a subprogramas (funções e sub-rotinas). A tabela é ainda pré-inicializada com as assinaturas das funções intrínsecas da linguagem (como MOD, ABS, SIN, SQRT, entre outras), facilitando a validação imediata de chamadas nativas.

4.2. Sistema de Tipagem e Declaração Implícita

O *Fortran 77* possui a particularidade histórica de suportar declaração implícita de variáveis. O analisador semântico implementa fielmente esta regra: caso uma variável não seja explicitamente declarada, o seu tipo é inferido com base na primeira letra do seu identificador (variáveis iniciadas pelas letras de I a N são categorizadas como INTEGER, sendo as restantes classificadas como REAL).

Para além da inferência, o analisador aplica verificações estritas de tipos nas expressões matemáticas e lógicas. Garante-se, por exemplo, que operadores aritméticos (+, -, *, /, **) apenas operam sobre tipos numéricos, e que os operadores lógicos (.AND., .OR., etc.) exigem operandos do tipo LOGICAL. Operações sobre *arrays* também são validadas, confirmando a declaração prévia das dimensões e atestando que os índices fornecidos são estritamente numéricos.

4.3. Validação de Controlo de Fluxo

Em resposta ao modelo de saltos do Fortran, implementou-se um mecanismo de registo e validação de *labels* (rótulos de linha). O método *_verify_and_reset_flow* garante a coerência estrutural do código, assegurando que:

- Qualquer salto GOTO tem como destino um rótulo efetivamente definido no âmbito atual;
- Os ciclos DO terminam obrigatoriamente na instrução CONTINUE correspondente ao rótulo exigido, prevenindo ciclos não fechados ou encadeamentos inválidos.

4.4. Validação de Subprogramas

Numa primeira passagem pela AST, o analisador regista as assinaturas de todos os subprogramas (`_register_signatures`). Posteriormente, ao encontrar nós de chamadas de funções (`FunctionCallExpr`) ou sub-rotinas (`CallNode`), o sistema cruza a aridade (número de argumentos fornecidos) com a definição original na tabela de símbolos, lançando exceções claras caso ocorram divergências.

5. Otimizações

Com o objetivo de **melhorar a eficiência** do código gerado e **eliminar redundâncias**, o compilador integra um módulo de otimização de código intermédio (`ASTOptimizer`). Este atua diretamente sobre a AST validada, aplicando múltiplas passagens de refinamento de forma iterativa, até que não ocorram mais alterações ou se atinja um limite máximo de iterações (configurado para 8). O processo divide-se em três eixos principais de atuação:

5.1. Simplificação de Expressões

Esta passagem otimiza operações matemáticas e lógicas em tempo de compilação, aliviando a carga computacional em tempo de execução:

- **Dobramento de Constantes (*Constant Folding*):** Operações binárias e unárias envolvendo apenas literais são pré-calculadas. Por exemplo, uma expressão como $2 + 3$ é substituída diretamente por um nó literal de valor 5;
- **Simplificações Algébricas:** Regras matemáticas universais são aplicadas para erradicar operações supérfluas. Exemplos concretos incluem a remoção de adições com zero ($x + 0$ transita para x), multiplicações por um ($x * 1$ resulta em x), e a resolução determinística de expressões booleanas (como $x \text{ .AND. } \text{TRUE}$, que é reduzido a x).

5.2. Eliminação de Código Morto

O otimizador realiza a poda ativa de caminhos de execução redundantes ou inalcançáveis:

- **Constantes em Condicionais:** Se a condição de uma instrução IF puder ser avaliada estaticamente (por exemplo, `IF ( .TRUE. )`), o compilador descarta o ramo ELSE e promove o bloco THEN, eliminando a instrução de salto condicional;
- **Corte por Inatingibilidade:** Qualquer instrução que suceda a um terminador de fluxo (como RETURN ou um comando nulo STOP) é sumariamente removida da lista de comandos.

5.3. Otimização de Saltos (*Jump Optimization*)

Dada a forte dependência do *Fortran 77* em saltos e rótulos (GOTO), implementou-se um algoritmo para reestruturação do grafo de controlo de fluxo:

- **Colapso de Cadeias de GOTO:** Se um GOTO direcionar a execução para outro GOTO, o analisador resolve a cadeia e aponta o primeiro salto diretamente para o destino final, poupando ciclos de máquina;
- **Remoção de *Fallthroughs*:** Comandos GOTO que apontam para a linha de código imediatamente a seguir (o fluxo natural de execução) são removidos por serem redundantes. Para assegurar a integridade do código, o algoritmo isenta desta transformação os rótulos que pertencem a ciclos D0, evitando a corrupção do motor de iteração.

5.4. Dificuldades Encontradas

A implementação do motor semântico e do otimizador de AST constituiu um desafio técnico considerável, derivado sobretudo das especificidades da especificação do *Fortran 77* e da natureza do paradigma de *parsing* utilizado. Entre os principais obstáculos, destacam-se:

- **Resolução de Ambiguidade de Escopo e Subprogramas:** A gestão de tabelas de símbolos para suportar sub-rotinas e funções exigiu um cuidado redobrado na transição de âmbitos. A necessidade de registar assinaturas de funções antes da sua visita completa (para permitir recursividade ou referências cruzadas) obrigou à implementação de uma passagem preliminar pela árvore, aumentando a complexidade do algoritmo de visita.
- **Validação de Rótulos de Fluxo:** Ao contrário das linguagens modernas, onde os blocos são delimitados por símbolos (como { } ou END IF), o *Fortran* depende de rótulos numéricos para o fecho de ciclos D0. A garantia de que todos os GOTO e D0 apontam para rótulos válidos e existentes no âmbito local revelou-se uma das tarefas mais árduas do analisador semântico, exigindo um sistema de registo e verificação pós-processamento para evitar erros de fluxo inatingível.
- **Preservação da Tipagem no “Constant Folding”:** No módulo de otimização, a implementação do *Constant Folding* exigiu uma lógica rigorosa para respeitar a precedência de tipos do *Fortran* (promoção de INTEGER para REAL). A simulação exata do comportamento da divisão inteira vs. divisão real em tempo de compilação foi fundamental para assegurar que o código otimizado produzisse resultados idênticos ao código original.
- **Prevenção de Ciclos Infinitos em Otimizações de Salto:** Durante o colapso de cadeias de GOTO, surgiu o risco teórico de entrar em ciclos infinitos caso o código fonte contivesse saltos circulares. Foi necessário implementar um mecanismo de deteção de visitas (conjuntos de nós visitados) para garantir que o otimizador terminasse sempre a execução, mesmo perante estruturas de código mal formadas.

6. Geração de Código Máquina

A fase de *Back-End* assume que a AST (composta por nós como `IfNode`, `DoNode` e `CallNode`) está completa e semanticamente validada. O objetivo desta etapa é traduzir estes nós para um formato executável focado numa arquitetura de máquina virtual baseada em pilha (Stack).

6.1. A Lógica da Stack e Offsets de Memória (FP)

Na máquina de pilha, cada subprograma necessita de um contexto de execução isolado, gerido através do **Frame Pointer (FP)**. Quando o `CallNode` é ativado, geram-se instruções para:

- Guardar o endereço de retorno;
- Atualizar o FP para a nova *stack frame*;
- Reservar *offsets* específicos locais para cada variável declarada.

As variáveis declaradas (`VarDecl`) ficam indexadas em posições como `FP + 1`, `FP + 2`, garantindo isolamento semântico.

6.2. Tradução de Ciclos DO e Estruturas IF para Labels

O Fortran não trabalha com parênteses chaveta, mas sim com marcas de salto (`GOTO` e `CONTINUE`). Ao processar um `DoNode`, a geração de código materializa a lógica através de labels de controlo:

```
START_DO_L1:
    push(var)          // Carrega iterador
    push(end)          // Carrega limite
    infere_condicao()    // Testa se iterador > limite
    jump_if_false END_DO_L1
    // Corpo do ciclo ...
    atualiza_var_step()
    jump START_DO_L1
END_DO_L1: (Mapeado para o CONTINUE do Label)
```

6.3. Lógica de Pass-by-Reference

Uma característica vital do Fortran 77 é que os argumentos passados para rotinas operam por **passagem por referência**. Na geração de código, o tradutor distingue literais de variáveis. Quando um parâmetro do tipo `Variable` é passado num `CallNode`, utiliza-se a instrução de extração de endereço (`pusha` em vez de `pushi`), permitindo que alterações dentro da rotina persistam na variável de origem.

7. Conclusão

O projeto resultou na construção de um **front-end** robusto para Fortran 77, validado com sucesso através da bateria de testes sintáticos e semânticos. Destaca-se a arquitetura modular baseada numa Árvore de Sintaxe Abstrata (AST) limpa, que separou eficazmente a análise de fluxo, expressões lógicas e a Tabela de Símbolos, garantindo um código escalável e de fácil manutenção.

Como trabalho futuro e consolidação do tradutor, o fluxo de compilação seria rematado com um módulo de **Back-End** (e.g., `compiler.py`). Este seria responsável por percorrer a AST validada e exportar as instruções finais da Máquina Virtual para um ficheiro objeto.

Em suma, o desenvolvimento deste compilador reforçou a importância do rigor absoluto no tratamento de ambiguidades gramaticais (como os conflitos *Shift/Reduce*) e evidenciou que uma divisão clara de tarefas arquitetónicas é vital para o sucesso do grupo.

8. Referências Bibliográficas

- [1] Universidade do Minho, «PL2026 - Construção de um Compilador para Fortran 77 Standard», 2026.
- [2] David Beazley, «PLY (Python Lex-Yacc) Documentation». [Em linha]. Disponível em: <https://www.dabeaz.com/ply/>
- [3] ANSI X3.9-1978, «American National Standard Programming Language FORTRAN», 1978.